

PasswordStore Smart Contract Security Audit

Author: Ibrahim Abdulkarim
Date: May 2, 2025
Audit Type: Smart Contract Security Assessment

Executive Summary

This report presents the findings of a security audit conducted on the PasswordStore smart contract. The audit identified multiple security vulnerabilities that significantly compromise the core functionality and security posture of the contract.

The PasswordStore contract is designed to store passwords securely on the blockchain, with access control mechanisms intended to restrict password retrieval to the contract owner. However, the audit revealed critical flaws in both the design approach and implementation that render the contract unsuitable for its intended purpose.

Audit Scope

The audit focused on the following contract:

- `PasswordStore.sol` (Solidity 0.8.18)

Key Findings

The audit revealed:

- **2 High-severity vulnerabilities** that completely undermine the security model of the contract
- **1 Informational issue** related to code documentation

Risk Classification

Severity	Impact	Description
High (H)	Critical	Issues that directly lead to loss of funds, data, or compromise of the system
Medium (M)	Significant	Issues that may lead to unexpected behavior but do not directly lead to loss of funds
Low (L)	Minor	Issues that are relatively small or do not present immediate danger
Informational (I)	Minimal	Non-critical issues, recommendations, or best practice suggestions

Contract Overview

The PasswordStore contract is a simple password storage solution with the following features:

- Storing a password string on the blockchain

- Setting a new password via the `setPassword` function
- Retrieving the password via the `getPassword` function (intended to be owner-only)

```
contract PasswordStore {
    error PasswordStore__NotOwner();

    address private s_owner;
    string private s_password;

    event SetNewPassword();

    constructor() {
        s_owner = msg.sender;
    }

    function setPassword(string memory newPassword) external {
        s_password = newPassword;
        emit SetNewPassword();
    }

    function getPassword() external view returns (string memory) {
        if (msg.sender != s_owner) {
            revert PasswordStore__NotOwner();
        }
        return s_password;
    }
}
```

Detailed Findings

[H-1] Storing password onchain makes it visible to anyone and no longer private

Description:

All data stored on the blockchain is publicly visible to anyone and can be read directly from the blockchain. The `PasswordStore:s_password` variable is intended to be a private variable and only accessed through the `PasswordStore:getPassword` function, which is intended to be called by the owner of the contract.

However, the `private` visibility modifier in Solidity only prevents other contracts from accessing the variable directly - it does not hide the data from being read from the blockchain itself. This is a fundamental misunderstanding of blockchain data visibility.

Impact:

Anyone can read the "private" password by directly accessing the blockchain storage, completely undermining the security model of the contract. This severely breaks the core functionality of the protocol, which is meant to store passwords securely.

Proof of Concept:

The following steps demonstrate how anyone can read the private password from the blockchain:

1. Create a local running node

```
make anvil
```

2. Deploy the contract onchain

```
make deploy
```

3. Read the password from contract storage

```
cast storage 0x5FbDB2315678afecb367f032d93F642f64180aa3 1 --rpc-url  
http://127.0.0.1:8545
```

4. Parse the bytes32 value to a string

```
cast parse-bytes32-string  
0x6d79506173737766726400000000000000000000000000000000000000000014
```

5. The output reveals the password

```
myPassword
```

Recommended Mitigation:

The entire password storage mechanism should be moved off-chain. Storing sensitive data on the blockchain is fundamentally insecure as all data can be read from the blockchain.

Potential alternatives include:

- Using a hash-based verification system where only password hashes are stored on-chain
- Implementing an encryption system with off-chain key management
- Using a completely off-chain solution for sensitive data storage with blockchain only for access control

[H-2] `PasswordStore::setPassword` has no access control, meaning a non-owner can set a password

Description:

The `PasswordStore::setPassword` function is intended to be called only by the owner of the contract, but it lacks any access control mechanism. This allows any external address to call the function and change the stored password.

```
function setPassword(string memory newPassword) external {  
    // Missing access control check  
    s_password = newPassword;  
    emit SetNewPassword();  
}
```

While the contract implements access control for the `getPassword` function, it fails to do so for the `setPassword` function, creating an inconsistent security model.

Impact:

Any address can set a new password, which completely undermines the security model of the contract. This allows malicious actors to overwrite the owner's password at any time, making the contract unreliable and insecure for its intended purpose.

Proof of Concept:

The following test case demonstrates how any address can set a password:

```
function test_anyone_can_set_password(address randomAddress) public {  
    vm.assume(randomAddress != owner);  
    vm.prank(randomAddress);  
    string memory expectedPassword = "myNewPassword";  
    passwordStore.setPassword(expectedPassword);  
  
    vm.prank(owner);  
    string memory actualPassword = passwordStore.getPassword();  
    assertEq(actualPassword, expectedPassword);  
}
```

Recommended Mitigation:

Add proper access control to the `setPassword` function to ensure only the owner can modify the password:

```
function setPassword(string memory newPassword) external {  
    if (msg.sender != s_owner) {  
        revert PasswordStore__NotOwner();  
    }  
    s_password = newPassword;  
    emit SetNewPassword();  
}
```

[I-1] There is no new password parameter in `PasswordStore::getPassword`

Description:

The NatSpec documentation for the `PasswordStore::getPassword` function incorrectly indicates a parameter that does not exist in the function signature.

```
/*
 * @notice This allows only the owner to retrieve the password.
 * @param newPassword The new password to set. // This parameter doesn't
exist
 */
function getPassword() external view returns (string memory) {
    // Function implementation
}
```

Impact:

The incorrect documentation may cause confusion for developers interacting with the contract. While this does not directly impact the security or functionality of the contract, it represents a documentation inconsistency that should be addressed.

Recommended Mitigation:

Remove the incorrect NatSpec line:

```
/*
 * @notice This allows only the owner to retrieve the password.
- * @param newPassword The new password to set.
 */
```

Security Considerations and Best Practices

Blockchain Data Visibility

- **Issue:** The fundamental design flaw in this contract is the assumption that data marked as **private** in Solidity is not accessible to external observers.
- **Reality:** All data on the blockchain is public and can be read by anyone with access to the blockchain.
- **Best Practice:** Never store sensitive information directly on the blockchain, even with **private** visibility modifiers.

Access Control

- **Issue:** Inconsistent application of access control across functions.
- **Best Practice:** Implement consistent access control for all state-changing functions, preferably using modifiers for clarity and to avoid code duplication:

```
modifier onlyOwner() {
    if (msg.sender != s_owner) {
        revert PasswordStore__NotOwner();
    }
    _;
}
```

Documentation

- **Issue:** Inaccurate NatSpec comments leading to confusion.
- **Best Practice:** Ensure all documentation accurately reflects the actual code implementation and is updated when code changes occur.

Conclusion

The PasswordStore smart contract contains critical security vulnerabilities that completely undermine its intended functionality. The most significant issues are:

1. **Fundamental Design Flaw:** Storing sensitive data (passwords) directly on-chain, making them publicly accessible despite the use of Solidity's `private` visibility modifier.
2. **Inconsistent Access Control:** Failing to implement access control on the `setPassword` function, allowing any address to modify the stored password.

These issues render the contract unsuitable for its intended purpose of secure password storage. A complete redesign of the approach is recommended, with consideration for off-chain storage of sensitive data and consistent application of access control mechanisms.

Recommendations

1. **Redesign the Password Storage Mechanism:**
 - Move sensitive data off-chain
 - Consider using cryptographic approaches like zero-knowledge proofs for password verification without revealing the password
2. **Implement Consistent Access Control:**
 - Add access control to all state-changing functions
 - Consider using the OpenZeppelin Ownable contract for standardized ownership management
3. **Improve Documentation:**
 - Correct all NatSpec comments to accurately reflect function parameters and behavior
 - Add comprehensive documentation explaining the security model and limitations

Disclaimer

This audit report is not a security warranty, investment advice, or an approval of the PasswordStore project. No security audit can guarantee the discovery of all possible vulnerabilities. This report represents a time-limited assessment of the current state of the contract's implementation and its security posture.